Task 11: Electricity Bill from Meter Readings

Student name	[ENTER YOUR NAME]	Register number	[ENTER REGISTER NUMBER]
GitHub repository	https://github.com/[username]/CSA02-[RegisterNumber]-Assignment1/tree/main/Task11	Compile command	gcc -Wall -Wextra -std=c11 -O2 main.c -o task
Compiler	Online C compiler (screenshots); own benchmarks GCC 13.3.0	Date	26 October 2026

1. Task 11 - Electricity Bill from Meter Readings

1.1 Problem analysis

The program reads a household number, the previous meter reading and the current reading; validates them; converts them to units consumed; applies the tariff via `calculate_bill(units)`; separates the fixed charge; and prints an itemised bill. All money is kept in paise (`long long`) and printed with `%lld.%02lld`.

1.2 Data types, constants and preprocessor concepts

Item	Type / form	Reason
METER_MAX	Macro, value 65535 (from the message "between 0 and 65535")	A 16-bit meter dial; also the upper bound of the validation test
FIXED_CHARGE	Macro, 5000 paise (Rs 50.00, from output)	Single place to change the fixed charge
previous, current	Unsigned, read with %u	Readings cannot be negative; scanf return value checked
units	Result of calculate_units(previous, current)	313 for 1200 to 1513
total_bill, energy_charge	long long, paise	energy_charge = total_bill - FIXED_CHARGE (line 79)
Return code 1	Early exit from main	Invalid input stops the program before any calculation

1.3 Source code (visible lines 61-97)

```
printf("Enter current meter reading: ");
if (scanf("%u", &current) != 1)
{ printf("Invalid reading.\n"); return 1; }
if (previous > METER_MAX || current > METER_MAX)
{   printf("Readings must be between 0 and 65535.\n"); return 1; }
units = calculate_units(previous, current);
total_bill = calculate_bill(units);
energy_charge = total_bill - FIXED_CHARGE;
printf("Household Number : %u\n", household_no);
printf("Units Consumed   : %u\n", units);
printf("Energy Charge    : Rs %lld.%02lld\n",
       energy_charge / 100, energy_charge % 100);
printf("Fixed Charge     : Rs %lld.%02lld\n",
       FIXED_CHARGE / 100, FIXED_CHARGE % 100);
printf("Total Bill       : Rs %lld.%02lld\n",
       total_bill / 100, total_bill % 100);
return 0;
```

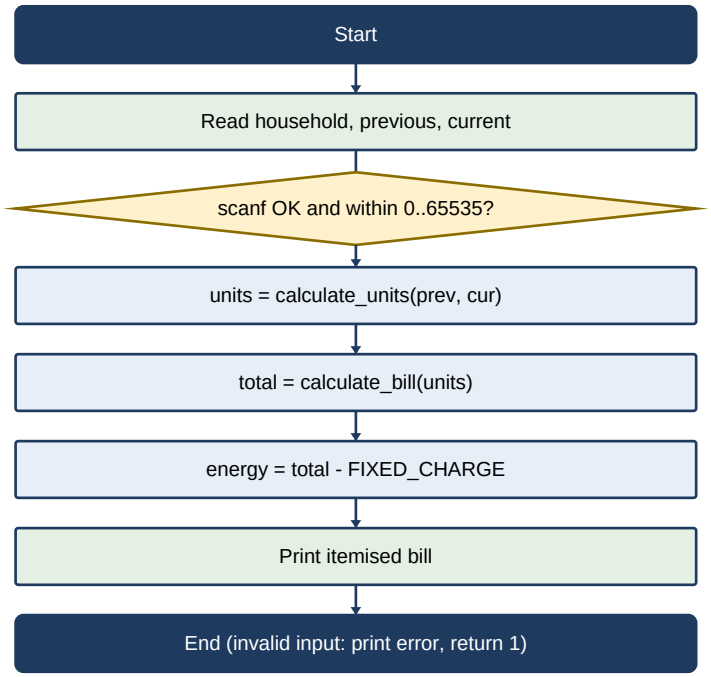
The bodies of `calculate_units` and `calculate_bill` and the tariff slabs were not visible in the screenshot, so the slab rates are not reproduced here. Paste them into section 1.3 from your source file.

1.4 Pseudocode

```
READ household_no, previous, current   (each scanf must return 1)
IF previous > METER_MAX OR current > METER_MAX THEN error, exit 1
units         <- calculate_units(previous, current)
total_bill    <- calculate_bill(units)          // paise, includes fixed charge
energy_charge <- total_bill - FIXED_CHARGE
```

```
        PRINT household, readings, units
        PRINT energy, fixed, total as Rs (x DIV 100).(x MOD 100)
```

UML activity diagram - Task 11



1.5 Output evidence

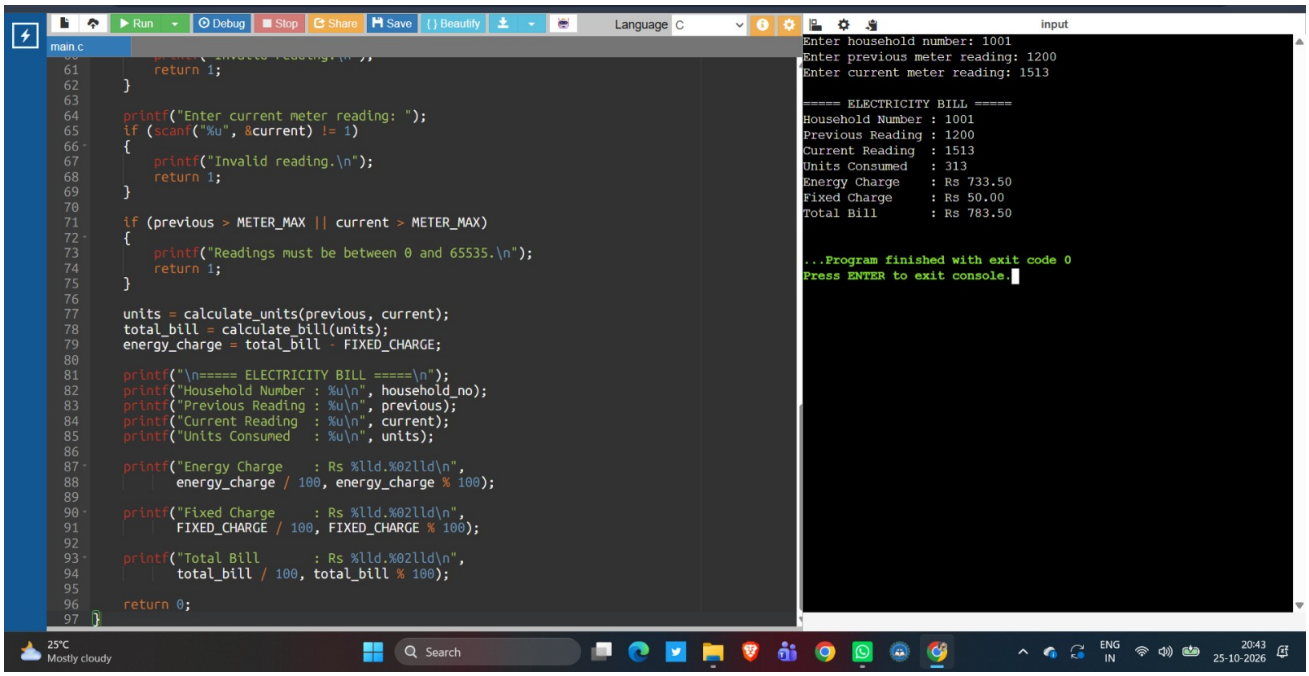


Figure 2 - Task 11 source and console output.

1.6 Test table (expected vs actual)

#	Input (household, prev, cur)	Expected	Actual	Result
1	1001, 1200, 1513 (normal)	Units = 1513 - 1200 = 313; total = energy + fixed; Rs 733.50 + Rs 50.00 = Rs 783.50	Units 313; Energy Rs 733.50; Fixed Rs 50.00; Total Rs 783.50	Pass

#	Input (household, prev, cur)	Expected	Actual	Result
2	Current reading 65536 (just above range)	"Readings must be between 0 and 65535." exit 1	[run and record]	[ ]
3	Previous 65535, current 65535 (upper boundary)	Accepted; 0 units; bill = fixed charge only (assuming no minimum charge)	[run and record]	[ ]
4	Non-numeric, e.g. "abc" for current	"Invalid reading." exit 1	[run and record]	[ ]
5	Current less than previous (rollover, e.g. 65500 to 20)	Depends on calculate_units wrap-around rule	[run and record]	[ ]

Only test 1 was in the screenshots; rows 2-5 are the boundary and invalid cases the assignment asks for and must be run on your machine, with a screenshot saved for each. Arithmetic check for test 1: 73350 + 5000 = 78350 paise, which prints as 783.50.

1.7 Alternative concept: table-driven slabs instead of an if-else chain

A chain such as `if (u <= 100) ... else if (u <= 200) ...` hard-codes each band. The alternative stores the bands in arrays and loops over them, so adding or changing a slab edits data, not logic.

```
typedef struct { unsigned upto; long long rate_paise; } Slab;
const Slab slabs[] = { {100, R1}, {200, R2}, {UINT_MAX, R3} };  /* R1..R3 = your rates */
long long bill = 0; unsigned prev_edge = 0;
for (size_t k = 0; k < sizeof slabs / sizeof *slabs && units > prev_edge; k++) {
    unsigned in_slab = (units < slabs[k].upto ? units : slabs[k].upto) - prev_edge;
    bill += (long long)in_slab * slabs[k].rate_paise;
    prev_edge = slabs[k].upto;
}
bill += FIXED_CHARGE;
```

	If-else chain	Table-driven loop
Time	O(1): at most s comparisons, s fixed	O(s) per bill, s = number of slabs (a constant, so O(1) in practice); O(n) for n households
Space	O(1)	O(s) for the table (a few bytes per slab)
Maintainability	Edit code for every tariff change	Edit data only; one loop to test

Both are O(1) per household when the slab count is fixed; the table version adds a small constant for loop overhead and gains flexibility. Billing n households is O(n) either way.